Java Interview Questions - Set 3

21. What is ASCII Code?

ASCII (American Standard Code for Information Interchange) is a 7-bit character encoding standard from 1963. It maps 128 characters to numeric codes (0-127).

Layout:

Range	Characters
0-31	Control characters (null, tab, newline, etc.)
32	Space
33-47	Punctuation (`!` , `"` , `#` , `$` , etc.)
48-57	Digits (`0` to `9`)
65-90	Uppercase letters (`A` to `Z`)
97-122	Lowercase letters (`a` to `z`)
127	DEL

Quick math you can do in your head:

- `'A'` = 65, `'a'` = 97 (difference of 32, so `'A' + 32 = 'a'`).
- `'0'` = 48, so digit conversion: `int n = c - '0';` .

In Java:

```
char c = 'A';
int code = c;              // 65, char auto-promotes to int
char fromCode = (char) 66; // 'B'
```

Limitations:

- Only 128 characters. No support for accents (`é` , `ñ`), non-Latin scripts (Greek, Arabic, Chinese), or symbols (`€` , `→`).
- Extended ASCII variants stretched it to 256 characters with different code pages, which caused endless interop pain.

ASCII is a subset of Unicode and UTF-8, so it still works inside modern systems. Just don't assume text is pure ASCII.

22. What is Unicode?

Unicode is the universal character encoding standard. It assigns a unique number (called a code point) to every character in every writing system: Latin, Cyrillic, Arabic, Chinese, Japanese, emoji, math symbols, archaic scripts.

Key facts:

- Code points are written as `U+XXXX` (hex). `U+0041` is `A`. `U+1F600` is 😀.
- Unicode defines over 150,000 characters across all known scripts.
- Code points range from `U+0000` to `U+10FFFF`, around 1.1 million possible characters.
- Unicode is the standard. UTF-8, UTF-16, UTF-32 are the encodings (how code points get stored as bytes).

Common encodings

Encoding	Bytes per code point	Common use
UTF-8	1-4 (variable)	Web, files, modern APIs
UTF-16	2 or 4	Java internal `char`, Windows APIs
UTF-32	4 (fixed)	Rare, simple but space-heavy

Java and Unicode

Java's `char` is a 16-bit UTF-16 code unit. ASCII characters fit in one `char`. Characters outside the Basic Multilingual Plane (BMP), like most emoji, need a surrogate pair (two `char`s).

```
String s = "😀";
System.out.println(s.length());                          // 2 (surrogate pair)
System.out.println(s.codePointCount(0, s.length()));     // 1
```

To iterate over real characters, use `codePoints()`:

```
"héllo 😀".codePoints().forEach(cp ->
    System.out.println(Integer.toHexString(cp))
);
```

ASCII is the first 128 code points of Unicode, so pure ASCII text is also valid Unicode text.

23. Difference between Character Constant and String Constant in Java?

Two literal types that look similar but behave differently.

Aspect	Character constant	String constant
Syntax	Single quotes: `'A'`	Double quotes: `"A"`
Type	`char` (primitive, 16-bit)	`String` (object)
Content	Exactly 1 Unicode character	0 or more characters
Stored as	A 16-bit integer code unit	An immutable String object on the heap, often in the string pool
Empty value	Not allowed (`''` is a compile error)	Allowed (`""` is the empty string)
Concatenation with `+`	Adds numerically	Concatenates as strings

Examples:

```
char c = 'A';   // character constant, value 65
String s = "A"; // string constant, a String object

char x = 'A';
int y = x + 1; // 66 (numeric addition)

String a = "Hello";
String b = "Hello";
System.out.println(a == b); // true, same pool entry

// Empty cases
// char empty = ''; // compile error
String empty = "";  // valid empty string
```

When mixed in `+`:

```
char c = 'A';
System.out.println(c + 1);       // 66 (int math)
System.out.println("" + c + 1);  // "A1" (string concatenation)
System.out.println(c + "1");     // "A1"
```

Use `char` for single-character handling (digit parsing, character comparison). Use `String` for any text longer than one character.

24. What are constants and how do you create them in Java?

A constant is a value that can't change after it's set. In Java, you create constants with the `final` keyword.

Instance constant

```
public class Config {
    private final int maxRetries = 3;
}
```

`maxRetries` can't be reassigned. It's set once during construction.

Class constant (the common case)

The canonical Java constant is `public static final`. By convention, the name is in `UPPER_SNAKE_CASE`.

```
public class HttpStatus {
    public static final int OK = 200;
    public static final int NOT_FOUND = 404;
    public static final int INTERNAL_SERVER_ERROR = 500;
}

// Usage
int code = HttpStatus.OK;
```

Breakdown of the modifiers:

- `public`: visible everywhere.
- `static`: belongs to the class, not instances.
- `final`: can't be reassigned after initialization.

Constants in an interface

Constants in an interface are implicitly `public static final`. The keywords are optional.

```
public interface PaymentLimits {
    int MAX_AMOUNT = 10_000;
    int MIN_AMOUNT = 1;
}
```

Same effect as `public static final` in a class. The "constants interface" anti-pattern (an interface that only holds constants) was common in older code. Modern Java prefers a final class with a private constructor or an `enum`.

enum for related constants

For a closed set of related constants, use an `enum`:

```
public enum Status {
    ACTIVE, INACTIVE, SUSPENDED, DELETED
}
```

Type-safe, exhaustive, supports methods. Reach for this when constants represent a fixed set of related values.

Compile-time vs runtime constants

A final field set with a compile-time expression (literals or other compile-time constants) becomes a compile-time constant. The compiler inlines its value at every use site.

```
public static final int MAX = 100;           // compile-time constant
public static final String NAME = "alice";   // compile-time constant

public static final long NOW = System.currentTimeMillis(); // runtime constant
```

Inlining matters for backward compatibility: if you change a compile-time constant in a library, dependent code needs to be recompiled to see the new value.

25. Difference between >> and >>> operators in Java?

Both shift bits to the right. They differ in how they handle the leftmost (sign) bit.

Operator	Name	Behavior
`>>`	Signed right shift	Preserves the sign. Fills the left with the original sign bit.
`>>>`	Unsigned right shift	Ignores the sign. Always fills the left with `0`.

For positive numbers, both produce the same result. The difference shows up with negative numbers.

Example with a positive number:

```
int x = 8;  // 0000 0000 0000 0000 0000 0000 0000 1000

System.out.println(x >> 2);   // 2 (0000 0000 ... 0000 0010)
System.out.println(x >>> 2);  // 2 (same)
```

Example with a negative number:

```java
int y = -8;  // 1111 1111 1111 1111 1111 1111 1111 1000

System.out.println(y >> 2);   // -2 (1111 1111 ... 1111 1110), sign preserved
System.out.println(y >>> 2);  // 1073741822, treated as a huge positive
```

When to use which:

- `>>` for arithmetic on signed values (dividing by powers of 2 while preserving sign).
- `>>>` when treating the bits as a raw pattern and the sign doesn't matter (hashing, bit packing, unsigned protocols).

Java has no `<<<` because left-shifting always fills with 0 regardless of sign. So `<<` is the only left shift.

26. Java coding standards for classes

Conventions developers follow so other developers can read the code. These rules come from Sun's original Code Conventions, and most are still enforced by Google Java Style and modern IDE defaults.

- Class names use `UpperCamelCase` (also called `PascalCase`). Examples: `OrderService`, `UserRepository`, `HttpClient`.
- Names are nouns. A class represents a thing. Action names belong on methods.
- One top-level public class per file. The file name matches the class name exactly, with a `.java` extension.
- Acronyms are camelCased. `HttpClient`, not `HTTPClient`. `JsonParser`, not `JSONParser`. Google style enforces this; some older codebases still go all caps.
- Use specific names. `OrderRepository` beats `DataManager`.
- Avoid prefixes like `C` or `T`. That's a C++ / Hungarian notation hangover. Java doesn't need it.
- One class, one responsibility. If the class name has "And" in it, you have two classes hiding in one.
- Source file order: package statement, imports, class declaration. Most styles avoid wildcard imports.
- Limit class size. When a class grows past 300-500 lines, it's usually doing too much.

Example:

```java
package com.example.orders;

import java.time.Instant;
import java.util.List;

public class OrderService {
    private final OrderRepository repository;
    private final EmailSender emailSender;

    public OrderService(OrderRepository repository, EmailSender emailSender) {
        this.repository = repository;
        this.emailSender = emailSender;
    }

    public void place(Order order) {
        repository.save(order);
        emailSender.send(order.customerEmail(), "order confirmed");
    }
}
```

27. Java coding standards for interfaces

Most class rules apply to interfaces too. A few interface-specific points:

- Interface names use `UpperCamelCase` like classes.
- Names usually describe capability. Common suffixes: `-able`, `-ible`, `-er`. Examples: `Runnable`, `Comparable`, `Serializable`, `Closeable`, `Iterable`, `EmailSender`.
- Don't prefix with `I`. `IUserRepository` is C# style. Java idiom is `UserRepository`. The implementation gets a more specific name like `JpaUserRepository` or `InMemoryUserRepository`.
- Methods are implicitly `public abstract`. Don't write the modifiers. They add noise.
- Fields are implicitly `public static final`. Avoid putting constants in interfaces. Use a final class or an `enum` instead.
- Prefer narrow interfaces. Interfaces with one method are easy to mock, easy to implement, and easy to compose. (See ISP in `java-solid-principles.pdf`.)
- Use `@FunctionalInterface` when you intentionally declare a single-abstract-method interface that lambdas should bind to.

Example:

```java
public interface EmailSender {
    void send(String to, String subject, String body);
}

@FunctionalInterface
public interface Validator<T> {
    boolean isValid(T input);
}
```

28. Java coding standards for methods

- Method names use `lowerCamelCase`. Examples: `getUserById`, `saveOrder`, `calculateTotal`.
- Names are verbs or verb phrases. A method does something. Use a verb form: `calculateTotal()`, `saveOrder()`, `validateInput()`.
- Keep methods short. One screen (around 30-50 lines) is a reasonable upper bound. Longer methods are usually doing too much.
- Each method does one thing. If you have to explain a method with “and” between two clauses, it’s two methods.
- Limit parameters. 3 or 4 is the soft ceiling. Beyond that, consider a parameter object.
- Boolean getters use `is`, `has`, or `can` prefix. `isActive()`, `hasPermission()`, `canDelete()`.
- Standard accessors are `getX()` and `setX()`. JavaBeans convention. Frameworks (Spring, Jackson, JPA) depend on it.
- Use `@Override` on every overridden method. Catches typos and signature mismatches at compile time.
- Don’t repeat the class name in method names. `user.getName()` reads better than `user.getUserName()` since the class is already `User`.
- Document public methods with Javadoc. Especially the contract: parameters, return value, thrown exceptions, side effects.

Example:

```java
public class OrderService {
    /**
     * Places an order for the given customer.
     *
     * @param customer the customer placing the order
     * @param items    the items in the order, must not be empty
     * @return the placed order with a generated ID
     * @throws IllegalArgumentException if items is empty
     */
    public Order place(Customer customer, List<Item> items) {
        if (items.isEmpty()) {
            throw new IllegalArgumentException("items must not be empty");
        }
        Order order = new Order(customer, items);
        repository.save(order);
        return order;
    }

    public boolean canPlace(Customer customer) {
        return customer.isActive() && !customer.isBlocked();
    }
}
```

29. Java coding standards for variables

- Variable names use `lowerCamelCase`. Examples: `userCount`, `firstName`, `isAvailable`.
- Names describe content. Use `users` instead of `userList`. The type is already in the declaration.
- Use full words. `temperature` reads better than `temp`, unless the variable lives for one line.
- Single-letter names are reserved for loop counters or short scopes. `i`, `j`, `k` for indices. `x`, `y`, `z` for coordinates. Anything else, spell it out.
- Boolean names sound like questions. `isActive`, `hasChildren`, `canEdit`. Avoid `flag` or `status` alone.
- Don't reuse names with different meanings across scopes. Confusing for everyone.
- Initialize at declaration when possible. `int count = 0;` beats declaring `int count;` then assigning later.
- Limit scope. Declare variables as close to where they're used as possible. Block-scoped beats class-scoped when one will do.
- Prefer `final` for locals that don't get reassigned. Helps both the reader and the JIT.
- Use `var` for local variables when the type is obvious. `var users = new ArrayList<User>();` reads fine. `var x = service.find(id);` is too vague.

Example:

```java
public List<String> activeUserNames(List<User> users) {
    var names = new ArrayList<String>();
    for (User user : users) {
        if (user.isActive()) {
            names.add(user.getName());
        }
    }
    return names;
}
```

30. Java coding standards for constants

- Constant names use `UPPER_SNAKE_CASE`. Examples: `MAX_RETRIES`, `DEFAULT_TIMEOUT_MS`, `BASE_URL`.
- Always `static final`. Constants belong to the class, not instances, and can't be reassigned.
- Use `public static final` for shared constants and `private static final` for internal ones.
- Group related constants. Put them at the top of the class so they're easy to find.
- Prefer enums for fixed sets of values. Type-safe, exhaustive, and supports methods.
- Don't put constants in interfaces. That's the "constants interface" anti-pattern. Use a final class with a private constructor or an `enum`.
- Use underscores in numeric literals for readability. `1_000_000` reads better than `1000000`.

Example:

```java
public final class HttpClientConfig {
    public static final int DEFAULT_TIMEOUT_MS = 5_000;
    public static final int MAX_RETRIES = 3;
    public static final String DEFAULT_USER_AGENT = "MyApp/1.0";

    private HttpClientConfig() {
        // prevent instantiation
    }
}
```

For a closed set of related values:

```java
public enum Status {
    ACTIVE,
    INACTIVE,
    SUSPENDED
}
```

Reach for `enum` first when the constants represent a closed set of related values.

Quick reference

Question	One-line answer
ASCII	7-bit, 128 characters, mostly English
Unicode	Universal standard, code points up to `U+10FFFF`
Character vs String constant	`'A'` is a `char`, `"A"` is a `String`
Constants in Java	`public static final` or `enum`
`>>` vs `>>>`	Signed shift (keeps sign) vs unsigned shift (fills with `0`)
Class naming	`UpperCamelCase`, noun, one public class per file
Interface naming	`UpperCamelCase`, often `-able` / `-er` suffix, no `I` prefix
Method naming	`lowerCamelCase`, verb, one thing per method
Variable naming	`lowerCamelCase`, descriptive, scope-limited
Constant naming	`UPPER_SNAKE_CASE`, always `static final`